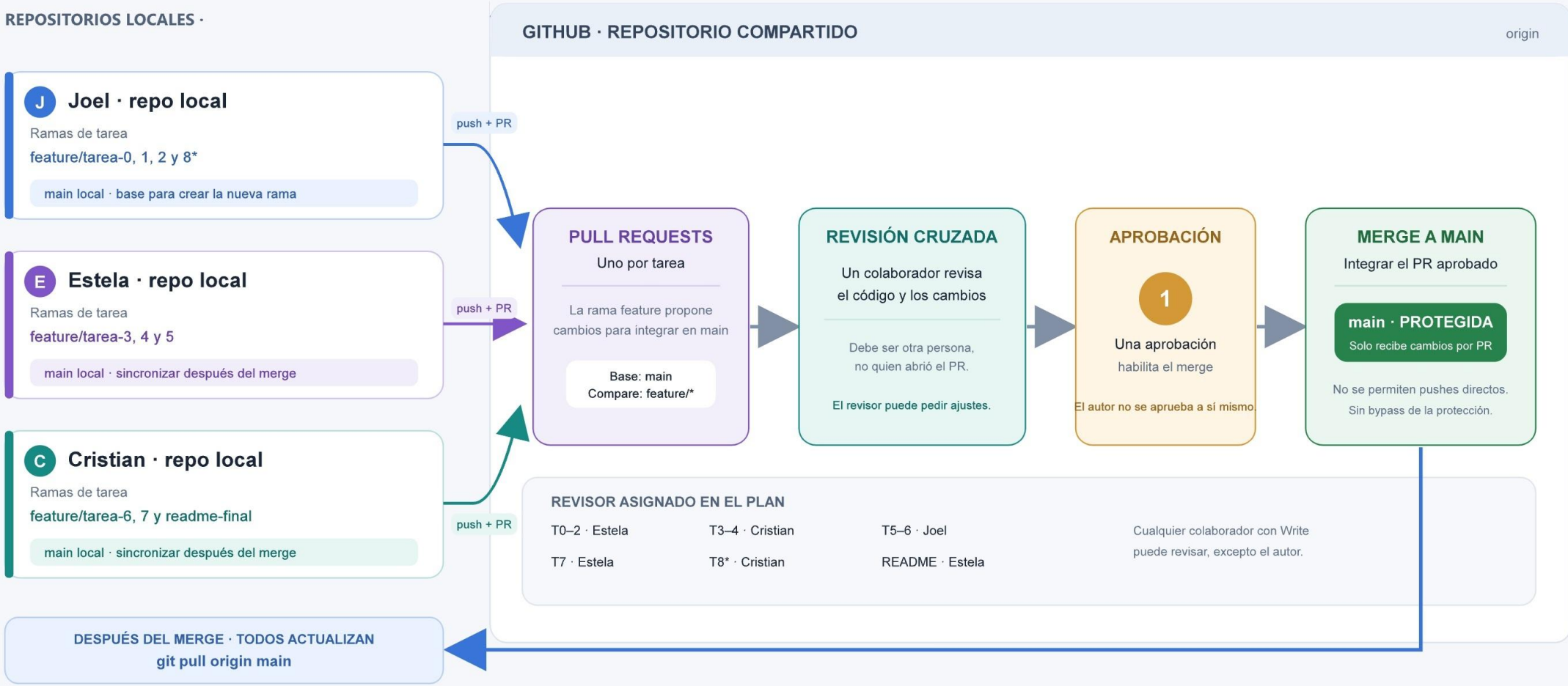
HANGMAN PROJECT · GIT Y REVISIÓN

De los repos locales a GitHub y de vuelta

Una tarea por PR



SECUENCIA

Tarea anterior mergeada en main → crear la siguiente rama

Tareas 0–7 obligatorias

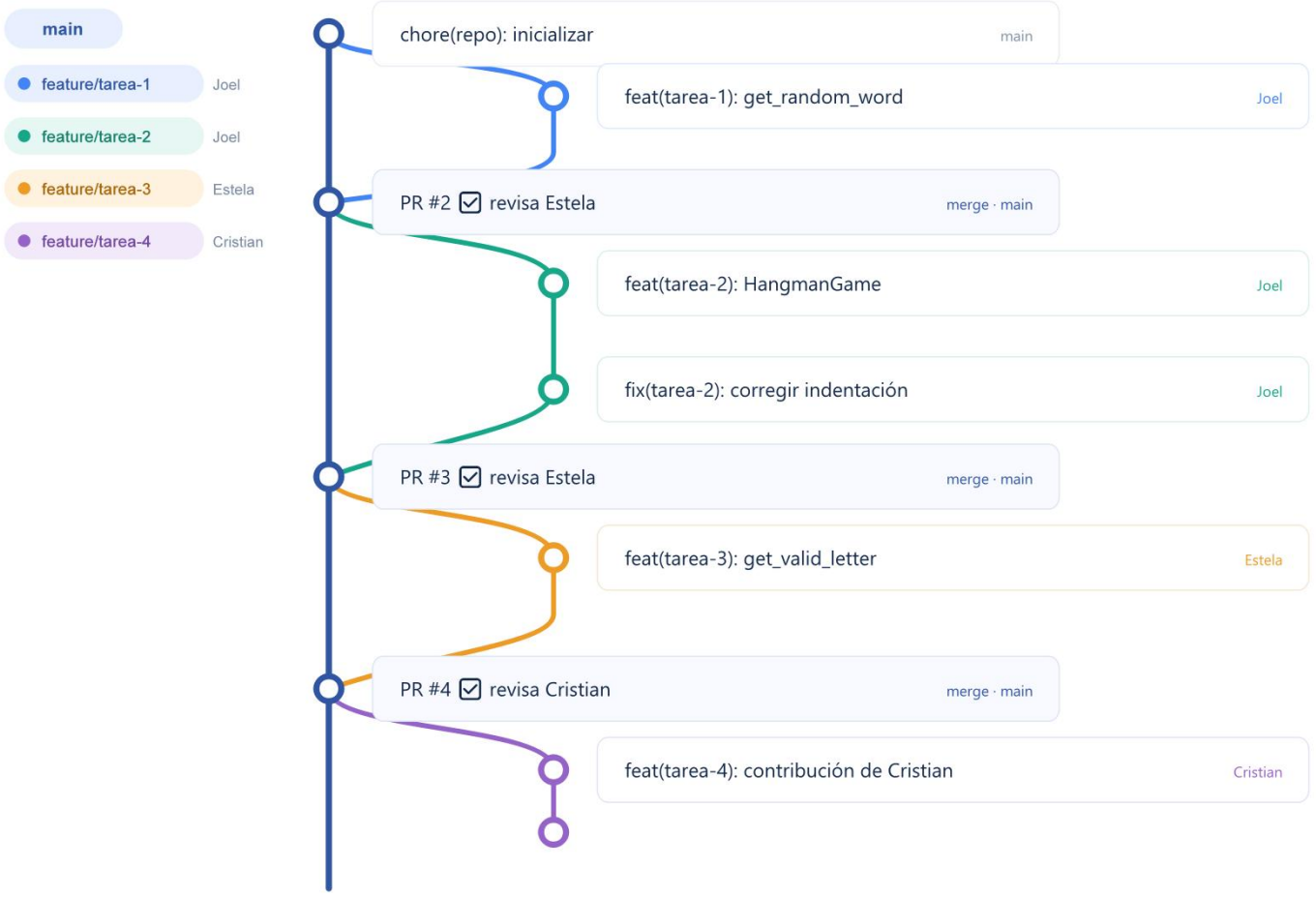
Tarea 8 opcional · README al cierre

Repositorio: github.com/jarciadca/hangman_project

Cristian prepara el README final · Estela lo revisa

Historial del repositorio

GitGraph · ramas de tareas y revisiones



FLUJO PRINCIPAL DE UNA PARTIDA

utils.py · get_random_word()

1. Leer wordlist.txt como UTF-8
2. Quitar espacios y pasar a minúsculas
3. Descartar líneas vacías o sin letras
4. Elegir una palabra o frase al azar

Si no hay palabras válidas → ValueError

utils.py · get_valid_letter()

Lee la entrada y aplica strip().lower().

La acepta solo si:

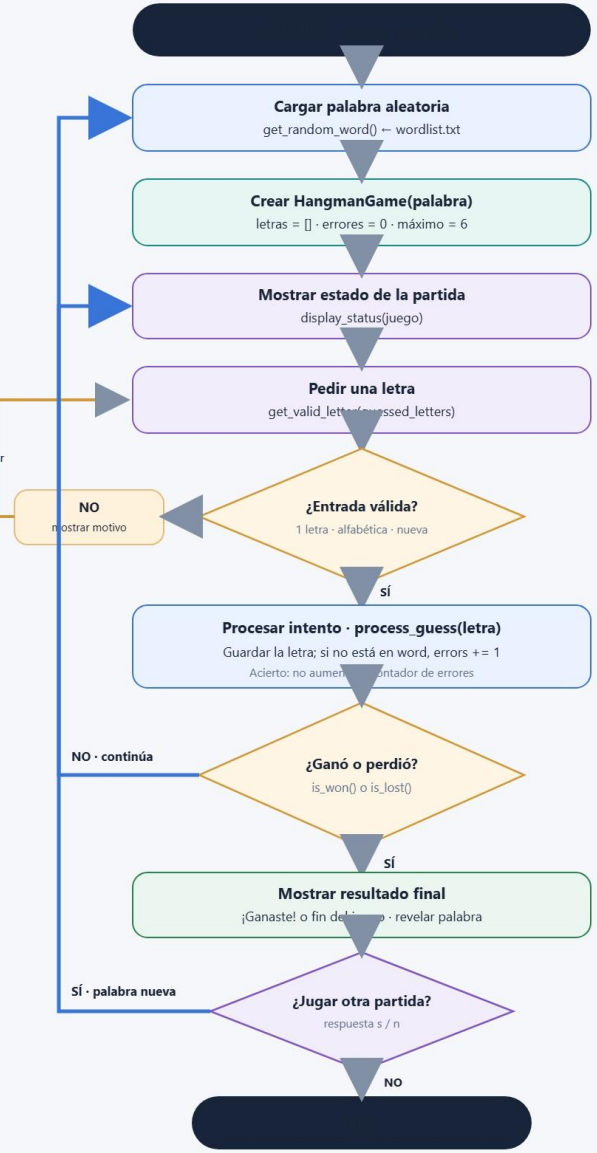
- tiene exactamente un carácter
- es una letra (isalpha())
- no se había intentado antes

Entrada inválida: explicar el motivo y repetir; no consume intento.

game.py · HangmanGame

word · palabra objetivo
guessed_letters · letras intentadas
errors · errores acumulados (empieza en 0)
max_errors · límite de errores: 6

Una partida = una instancia del juego



FUNCIONES Y REGLAS DEL CÓDIGO

game.py · proceso del intento

- Registrar la letra en guessed_letters
- Si aparece en word: acierto
- Si no aparece: sumar un error
- Máximo: 6 errores

La validación bloquea letras repetidas

utils.py · display_status()

- Dibujo del ahorcado según errores
- Progreso de la palabra
- Letras intentadas y errores restantes

Espacios y guiones se revelan solos
no consumen intentos ni impiden ganar

Condiciones de fin

Victoria · is_won()
Adivinó todas las letras alfabéticas.

Derrota · is_lost()
errors ≥ max_errors (6).

La partida termina al cumplirse cualquiera de las dos.

¿Qué hace cada archivo?

main.py	coordina la partida y el replay
game.py	guarda estado y reglas del juego
utils.py	carga, valida y muestra información
wordlist.txt	palabras y frases candidatas

El flujo se repite hasta ganar, perder o salir.

main.py coordina · game.py decide · utils.py ayuda · wordlist.txt aporta la palabra

README.md | main.py | utils.py | game.py

proyecto_ahorcado > hangman_project > main.py > jugar_una_partida

```python
"""Punto de entrada del juego del ahorcado."""

from utils import get_random_word, get_valid_letter, display_status
from game import HangmanGame


def jugar_una_partida():
    """Ejecuta una partida completa y muestra el resultado."""
    palabra = get_random_word()
    juego = HangmanGame(palabra)

    while not juego.is_won() and not juego.is_lost():
        display_status(juego)
        letra = get_valid_letter(juego.guessed_letters)
        acierto = juego.process_guess(letra)
        if acierto:
            print(f"¡Bien! La letra '{letra}' está en la palabra.")
        else:
            print(f"Fallaste. La letra '{letra}' no está.")

    display_status(juego)
    print()
    if juego.is_won():
        print(f"¡Ganaste! La palabra era: {juego.word}")
    else:
        print(f"Fin del juego. La palabra era: {juego.word}")
```

Problems | Output | Debug Console | Terminal | Ports 17

```
→ hangman_project git:(main) python3 main.py
Ingresa una letra: 5
Error: debes ingresar una letra.
Ingresa una letra: av
Error: debes ingresar exactamente una letra.
Ingresa una letra:
```

README.md | main.py | utils.py | game.py

proyecto_ahorcado > hangman_project > main.py > jugar_una_partida

```python
def main():
    """
    Punto de entrada con bucle de replay.

    Envuelve una partida en jugar_una_partida() y pregunta si repetir.
    """
    while True:
        jugar_una_partida()
        # .strip().lower() normaliza para aceptar "S", "s", " s ".
        respuesta = input("¿Jugar otra partida? (s/n): ").strip().lower()
        # Solo salimos si escribió 'n'. Cualquier otra cosa reinicia.
        if respuesta == "n":
            print("¡Gracias por jugar!")
            break


if __name__ == "__main__":
    main()
```

Problems | Output | Debug Console | Terminal | Ports 17

```
→ hangman_project git:(main) python3 main.py
Ingresa una letra: 5
Error: debes ingresar una letra.
Ingresa una letra: av
Error: debes ingresar exactamente una letra.
Ingresa una letra:
```

Desiciones técnicas importantes.

Patrón estándar de Python para puntos de entrada

```
if __name__ == "__main__":
    main()
```

Importable: si otro módulo hace import main, no se ejecuta el juego.
Testeable: los tests pueden importar main sin lanzar el juego.

Delegación total a game y utils (arquitectura)

```
pythonpalabra = get_random_word()          # ← utils
juego = HangmanGame(palabra)               # ← game
display_status(juego)                      # ← utils
letra = get_valid_letter(...)              # ← utils
acierto = juego.process_guess(letra) # ← game
```